



华南虎视觉组梯队任务

姓名：柯书凡 git仓库：<https://github.com/Samurai-aaa/27-vision.git>（私有）

核心任务

1. 装甲板自瞄

完成情况：全部必做+PnP进阶+EKF部分进阶

1.1. 代码思路

为了可以完整体现整个任务要求，我将三点任务分为三个task：

- task1_armor_detection：装甲板检测模块，内含传统视觉检测和神经网络识别检测
- task2_PnP：PnP检测模块，装甲板检测继承task1的神经网络识别检测，然后增加PnP解算算法。docs是关于几种PnP的原理笔记和使用对比
- task3_KF：卡尔曼滤波模块，继承task2的全部算法，加上Tracker模块，并完整改写成ROS2通信模式。包含simple_tracker和tracker两个节点，分别发布有cv、ca两个模式的KF和EKF追踪信息

- task1：传统识别：先对传入帧进行二值化，然后检测并匹配灯条对，组成一个装甲板并绘制
神经网络识别：通过本地部署深大神经网络模型，直接对网络输出的角点进行筛选和绘制
- task2：docs：多种PnP解算方法的笔记和使用情况对比分析 PnP：内含IPPE单解，双解，以及其他PnP解算，并对z轴进行绘制，同时筛选双解 PnP进阶：研究侧 = 固定xyz对yaw进行一维优化, 用 costSq/costL1 画代价曲线考察单/多谷、双解歧义、近正对跳变,并对 grid 等五种优化器比准确率与耗时;落地侧 = 重投影误差>6px 才在 $\pm 15^\circ @ 0.5^\circ$ 内遍历取误差最小
- task3：math_tools、extended_kalman_filter:滤波内核与数学工具(EKF/KF 共用;不传非线性钩子即退化为普通 KF) simple_target、simple_tracker：对单块装甲板进行普通卡尔曼滤波追踪（不包括整车预测、可以切换CA和CV两种模式） target、tracker：对整车及装甲板进行扩展卡尔曼滤波追踪

1.2. 遇到问题

1. 使用EKF强制预测TEMP_LOST，在装甲板旋转到背侧时，预测装甲板检测框会往屏幕右侧飞去，而不是绕车心轴进行旋转
2. 在实现稳定追踪装甲板时，容易出现跳变，无论是同一辆车还是同车的不同装甲板
3. IPPE双解最开始只用物理筛（筛选朝向为后的解），但是并无法完全筛选掉所有的错误解，或者会把两个同时筛掉
4. 尝试添加动态Q,但是一直无法调试好，待解决

1.3. 解决思路

1. 通过阅读代码，发现是在参考sp_vision代码时，误把小车的车心轴方向设置为相机坐标系的z轴，这在sp_vision里是没错的，因为sp_vision将小车的坐标轴移动映射到世界坐标系中，世界坐标系的z轴的确是竖直的，但是由于该项目仅处理视频，并没有进行相机坐标系到世界坐标系的转换，所以真正的竖直轴应该是y轴。
2. 跳变主要来自两处——整车若每帧从单板 PnP 结果重建会抖；同车换板瞬间锁定的板会在相邻装甲板间跳。全程只维护一个整车 EKF Target，状态跨帧延续、不重建整车；数据关联先按车号过滤只留目标车，再按板法线相位 φ 匹配观测对应的板；用 primary_id_ 锁住"正在追踪"的板，只有它转出视野才移交给下一块，并在车状态完全 LOST 前不切换/重建其他车。
3. IPPE 双解不能简单当成相对装甲板一前一后的镜像对——常规姿态下两解法线 z 分量同号、都朝前，只按"朝前/朝后"做物理筛分不开，极端情况还会把两个解一起筛掉。把两个解都重投影回图像并比较像素误差，在朝前解中取误差最小的解作为最终解（真解重投影 ≈ 0 ，镜像解误差明显更大）。

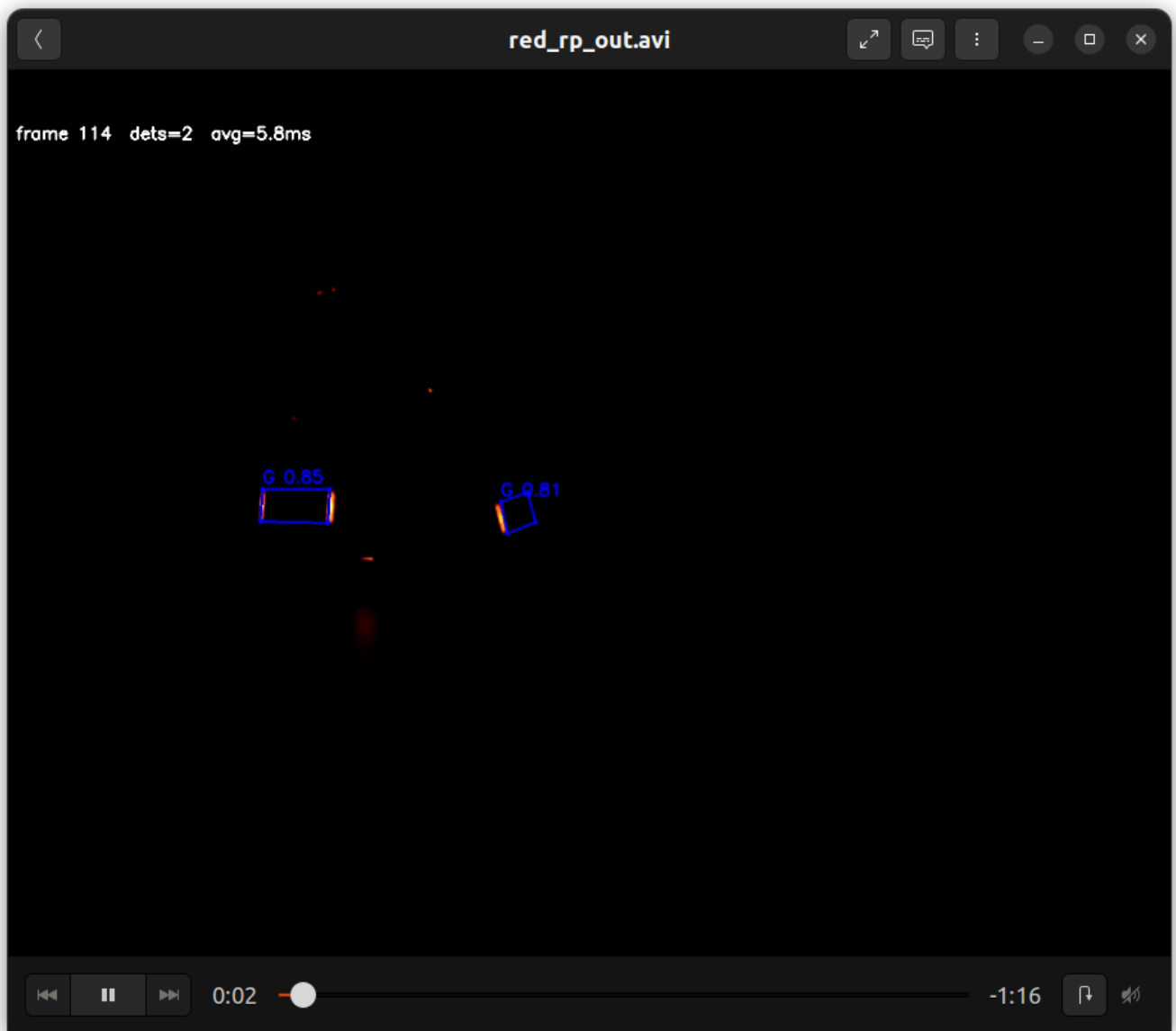
1.4. 效果图

- task1

传统视觉识别效果图：



神经网络识别效果图：



- task2

IPPE双解：

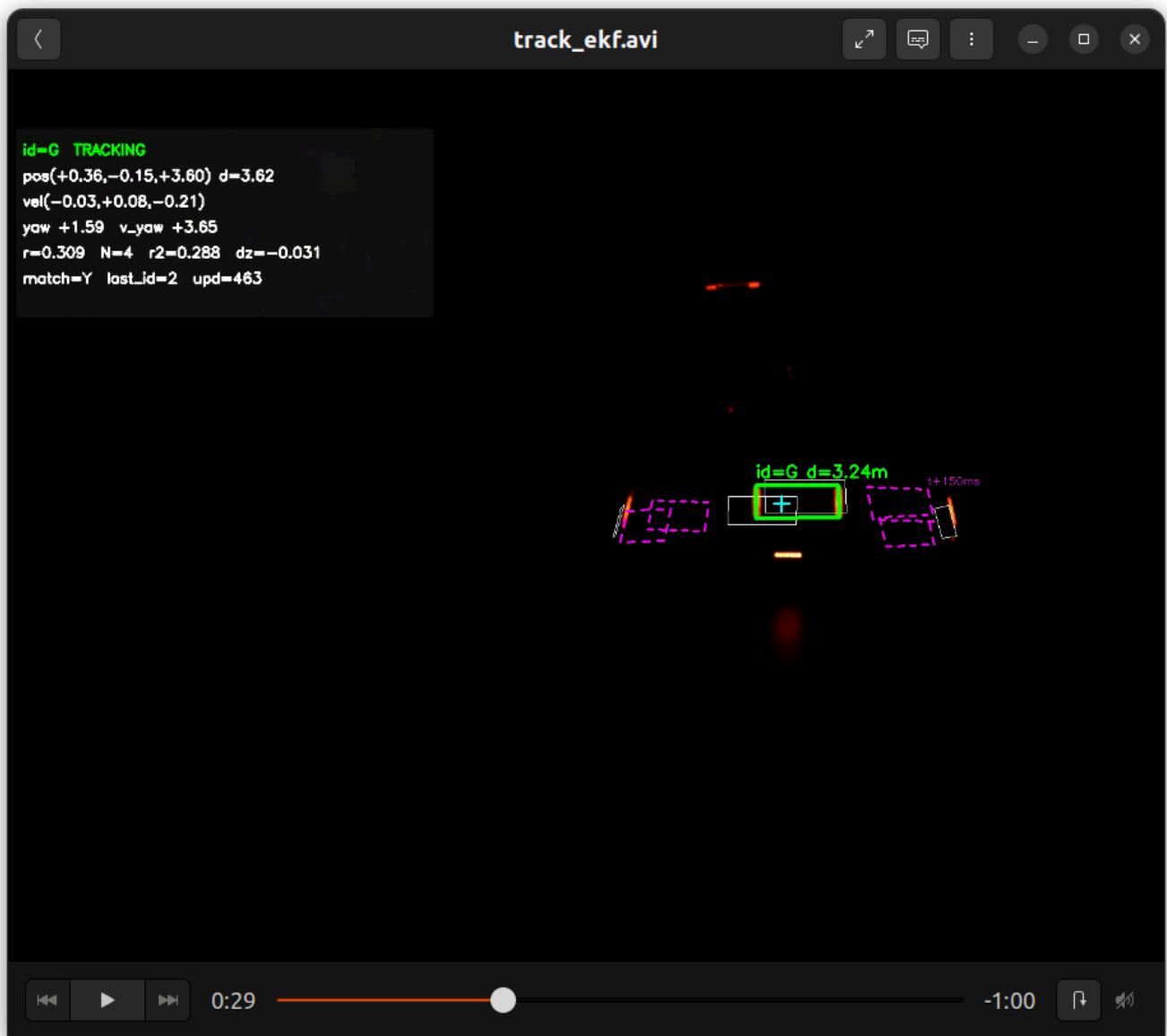


- task3

KF (CA)



EKF追踪



1.5. 总结

本项目虽然依旧是老样子的装甲板自瞄，但是项目任务中提出的任务也并非可以轻松完成的，在复现过程中也收

2. CMake框架搭建

工程按 task 分三层独立 CMake 工程，统一 C++17；三层无跨目录 include，代码与构建均可单独搬运。

- **task1 / task2 (普通单机 CMake)：** `find_package(OpenCV)`；NN 推理再 `find_package(OpenVINO)`（编译前需 `source /opt/intel/openvino_2024.6.0/setupvars.sh`，它把 `OPENVINO_DIR` 指到 `runtime/cmake` 才能被找到），链 `openvino::runtime`。task2 在同一工程内出两个目标：

主程序 `rp_detect` 与研究工具 `rp_yaw_study`（共享 `Openvinolnfer/armor/solver` 源，研究算法独立成 `yawstudy` 模块）。头文件 `target_include_directories(... PRIVATE include)`，产物进 `build/`。

- **task3（升级为 ROS2 colcon 工作空间，三个 `ament_cmake` 包）：**

- `armor_interfaces`：用 `rosidl_generate_interfaces` 生成自定义消息 `Armor/Armors/Target/TargetInfo`（依赖 `std_msgs/geometry_msgs`）；
- `detector`：`detector_node`（含 `video_player_node`）经 `ament_target_dependencies` 接 `rclcpp/sensor_msgs/cv_bridge/visualization_msgs` 等，链 `OpenCV+OpenVINO`；`Model/config/launch` 用 `install(DIRECTORY ...)` 装进 `share/`，节点经 `ament_index_cpp` 定位，摆脱对运行目录的相对路径依赖；
- `tracker`：把滤波算法抽成**不依赖 ROS 的静态库 `tracker_core`**（`extended_kalman_filter / simple_target / target / tracker`，只链 `Eigen`），`tracker_node` 与 `simple_tracker_node` 只是两个 ROS 壳（话题收发 + 2D 渲染）；`config/launch` 随包安装，供 `ros2 launch tracker tracker.launch.py` 使用。

- **分层取舍**：算法层（PnP/滤波）与 ROS 通信层解耦——同一份 `tracker_core` 同时支撑整车 EKF 与单板 KF 两个节点；构建层负责把模型、参数、`launch` 一并装包，节点运行时不再依赖当前工作目录。运行链路：`setupvars.sh` → `colcon build` → `source install/setup.bash` → `ros2 launch / scripts/start.sh`。

总结

由于有了之前的有关的复现经历本次项目总体比较顺利，算法效果不错，在开发过程中，整体流程阻碍並不算很：同时，对于ai的使用也有了更好的理解和把握。实际上我除了核心算法模块搭建之外的许多dirty work, 都是